Deck Speaker Script

Nov 12, 2026 · @dv

How to use this

A read-aloud script for the 7-slide TetrisFormer deck (`TetrisFormer.pptx`, generated by `make_deck.js`), about 8–10 minutes. Each slide has what to say, then the one thing listeners should walk away with.

The goal: two teammates leave knowing what we built, what the numbers mean, and where the results are weaker than the slide titles suggest.

Slide 1 — TetrisFormer (~1 min)

"The question we asked: can an agent learn to play a game well without anyone writing a reward for it? Our answer is to learn the reward from memory traces of good play.

We record the Game Boy's RAM while a competent player plays Tetris. A small transformer learns to predict the next RAM state. Then, when an RL agent plays, we reward it for producing RAM that the transformer finds likely — in other words, memory that looks like expert play.

Three things we never use: a hand-written reward, pixels, or action labels. The agent sees RAM and the reward model sees RAM.

The image shows three players on the same piece sequence: the scripted expert, our agent, and a curiosity agent."

Takeaway: reward = how expert-like the machine's memory looks.

Context if asked: this is a 2-day proof of concept. The long-term plan was Super Mario Land and Pokémon; Tetris was the quick testbed.

Slide 2 — The reward transformer (~1.5 min)

"Walk the four boxes left to right.

1. Every 24 frames we snapshot the 8 KB of work RAM. Most bytes never change, so we keep the 289 that do.
2. Each byte becomes a token: its value, plus an embedding for which address it is and which time step it's from. Width 128.
3. A causal transformer, 4 layers and 4 heads. The mask is block-causal: every byte at step t can see all bytes from steps up to t.

4. For every byte of step t+1, it outputs a 256-way softmax — a guess at that byte's next value.

It's tiny: 0.9 million parameters. A window is 8 steps × 289 bytes = 2,312 tokens.

The reward is the summed log-probability of the real next state. High means 'this is what an expert's memory would do next'."

Takeaway: it's a next-RAM-state language model; its log-likelihood is the reward.

Slide 3 — Training data (~1 min)

"Where does 'expert' come from? A scripted player: it searches every placement and scores boards with a standard linear Tetris heuristic. It plays 100 games and clears about 53 lines per game.

We keep only its RAM — 183k steps. Actions and pixels are thrown away.

Some bytes churn no matter what you press, like the sound driver and the stack. We mask those out so the model isn't rewarding noise.

Training is cheap: 3,000 AdamW steps, one-cycle LR peaking at 1e-3, batch 32, about 15 minutes on a laptop RTX 5060.

We split by whole games: 90 to train, 10 held out."

Takeaway: zero labels — just memory traces of a good player.

Slide 4 — Model quality (~1 min)

"On the 10 held-out expert games, the model reaches 0.040 nats per byte. Lower is better.

Compare with three baselines: random play scores 0.417, copying the last state 0.505, and a per-address frequency table 0.715.

So it's 12× better than just copying the last state. And random play is about 10× less likely per byte than expert play — that gap is exactly what makes it usable as a reward."

Takeaway: RAM is very predictable, and the model clearly tells expert from random.

Context if asked: a model trained without the noisy-byte mask reaches the same validation loss.

Slide 5 — The RL loop (~1.5 min)

"Follow the arrows around the diagram.

The PyBoy emulator runs Tetris and hands the policy the current RAM. The policy is a plain MLP over one-hot RAM; it picks a button.

The last 8 RAM states go to the reward transformer, which is frozen — we never update it during RL. It scores how likely the real next state was. We normalise that and add a small curiosity bonus (RND, weight $\beta = 0.5$) to help exploration.

PPO uses that reward to update the policy, and the loop repeats.

There are two action spaces. The button version has 8 actions. The placement version has 40: 4 rotations × 10 columns, carried out by the expert's own button logic."

Takeaway: standard PPO; the only new part is where the reward comes from.

Context if asked: we added a KL early-stop to PPO after one run collapsed into pressing a single button forever.

Slide 6 — Button-level results (~1.5 min)

"After 3 million steps, we score each player by how unlikely its RAM is under the reward model. Lower is more expert-like.

Expert 5, our agent 16, the hand-reward agent 40, random 106. The order is what you'd hope for.

The right panel is the more interesting result. Both hand-written rewards got hacked: the agent learned to pause the game. Our learned reward scores pausing as the worst behaviour of all. When we flip its sign to make it a curiosity reward, the agent pauses 94% of the time."

Takeaway: the learned reward resists the pause exploit that broke the hand-written ones.

Be upfront about:

- The ranking uses the same model that trained our agent, so 'ours beats hand' is partly circular. The expert and random endpoints are not.
- 3 seeds for ours and for hand, on unseen game seeds.
- At button level, every agent still clears about 0 lines per game ($\leqslant 0.08$).

Slide 7 — Placement actions and takeaways (~1.5 min)

"Button-level exploration was the bottleneck, so we gave agents 40 placement actions instead and trained each for 1 million placements.

Under our reward, per placement: expert −11.1, hand-reward agent −25.4, our agent −28.0, random −30.7. The reward still orders expert above agents above random.

Note the hand-reward agent scores better than ours on *our own* reward. So the reward is pointing the right way; PPO just didn't optimise it well. The signal is dense but weak and noisy.

Four takeaways: RAM is easy to model. The reward needs no labels and ranks behaviour sensibly. Every gap traces back to a few specific RAM bytes, so it's interpretable. And it's cheap: 1M placements took about 4 hours on a rented RTX 5090."

Be upfront about:

- Lines per game: hand-reward agent 0.71, ours 0.08, random placements 0.0, expert 59.
- One seed each.
- The placement actions bake in Tetris knowledge, which weakens the 'no game-specific engineering' claim.

Likely questions

Question	Short answer
Does our agent actually play Tetris well?	No. About 0 lines per game at button level, 0.08 with placement actions. The result is that the reward ranks behaviour correctly, not that the agent is good.
Why mean log p per placement, not the sum?	A sum favours high stacks: pieces fall for fewer frames, so there are fewer negative terms.
What was the pause exploit?	The hand-reward agents learned to press Start and sit paused. Our reward scores pausing worst.
Why the KL guard?	Two of three unguarded runs collapsed into always pressing one button. The guard doesn't fully prevent collapse (the no-mask run with it still collapsed).
Why did the placement run move to a 5090?	The laptop run was lost to a reboot at 525k placements. The rented-GPU rerun matched the laptop's first update exactly.
What's next?	Better optimisation of the learned reward, more seeds, and human Tetris play as training data.

Where to look in the repo

- `REPORT.md` / `REPORT.pdf` — full write-up
- `RESULTS.md` — numbers by run
- `DEVIATIONS.md` — every change from the plan and why (D-009 to D-012 cover the results above)
- `notes/log.md` — day-by-day events, collapses, ops issues
- `rm.py` (reward model), `ppo.py` / `rl.py` (RL), `expert.py` (scripted player), `make_deck.js` (this deck)